DO NOW — RECALL

Thinking procedurally: ordering subprocedures

Name: _____ Class: _____ Date: _____

1. Why does `age>12 && age<18` reject two valid ages for a range 12–18 inclusive?

2. What is a subprocedure?

3. What is an object's state?

4. Why not send a success message before saving a transaction?

DO NOW — CHECK / OBJECTIVES

Correct and improve

Check page 1 in GREEN PEN. Write corrections in full.

1. Model answer

Because strict comparisons exclude both 12 and 18.

2. Model answer

A named, reusable piece of a larger task.

3. Model answer

The stored values of its properties or fields.

4. Model answer

The save might fail even though the user has already been told it succeeded.

Vocabulary — know these terms

decomposition — Split a problem into smaller operations.

subprocedure — Named step used within a larger solution.

dependency — Stage that must precede another.

state update — Change the stored record or value.

transaction — Coordinated operations that must succeed reliably.

WE DO — SOURCE CHECK / IMPROVE

Only use this section AFTER attempting the We Do, You Do and Exit Ticket. Compare with the original classroom lesson's CHECK explanation. Correct errors in GREEN PEN and explain why.

Guided 1 — original CHECK

Model the reasoning

Store the loan and change availability, then confirm only if those actions succeed.

IMPROVE: Explain one correction to your own reasoning.

We Do Check · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

Order steps for borrowing a library book online.

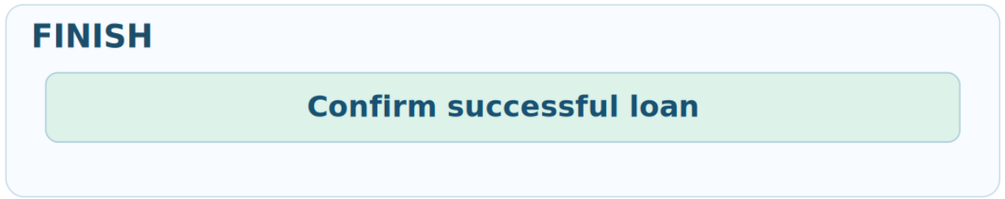
Identify the book and verify availability before trying to record a loan.



UPDATE; Record borrower; Mark unavailable

Order steps for borrowing a library book online.

Identify the book and verify availability before trying to record a loan.



FINISH; Confirm successful loan

NEW LEARNING

Read, annotate and explain

Decompose the task

Separate the problem into input validation, checking availability, updating the record and notifying the user.

Identify dependencies

No booking should be saved until seats are available; no confirmation should be sent before a successful save.

Make smaller operations reusable

ValidateRequest and SendConfirmation can become independently testable subprocedures.

Explain one step in your own words, using the correct technical terms.

New Learning · original lesson diagrams

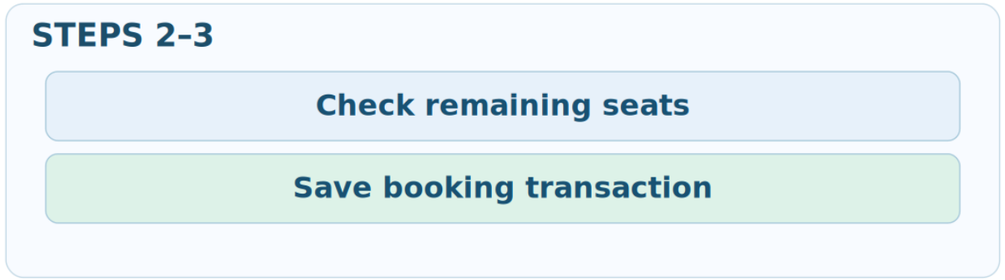
Diagrams appear after the related explanation. Annotate the original source visual.

New Learning



STEP 1; Read number of seats; Check request is valid

New Learning



STEPS 2–3; Check remaining seats; Save booking transaction

New Learning



STEP 4; Send confirmation AFTER save

EXIT TICKET / REFLECTION

Check your understanding

Complete the actual lesson Exit Ticket before checking the answer.

A booking confirmation can safely be sent before the record is saved.

How confident are you? Circle one: Confident / Getting there / Need help

Explain your choice, identifying one concept you can now use independently.

GREEN PEN — Correct and explain

Choose one mistake from your independent work. Write the correct method and explain precisely why your original answer was wrong.

Next step: revisit the model or a relevant revision-guide section if needed.

YOU DO — INDEPENDENT PRACTICE

Questions · Part 2

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

Question 3

A booking code subtracts seats from a local int parameter but never updates the central system. Identify three problems and three remedies.

I DO — WORKED EXAMPLE

Follow the worked method

Put password-reset stages in a safe order.

Predict first

First identify the account and create a single-use reset token.

Model the reasoning

Check the token before accepting a new password.

Model the reasoning

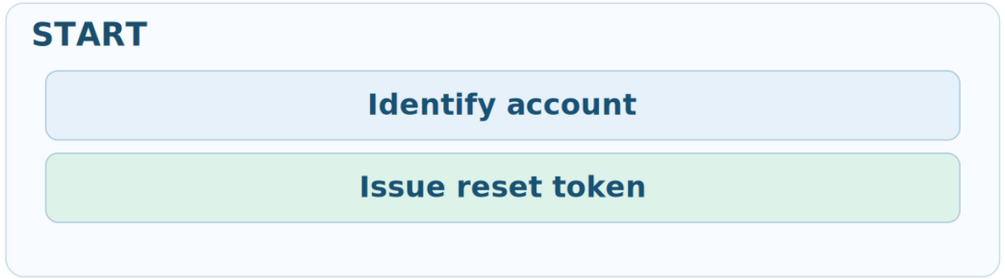
Save the change successfully before confirming to the user.

I Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

Put password-reset stages in a safe order.

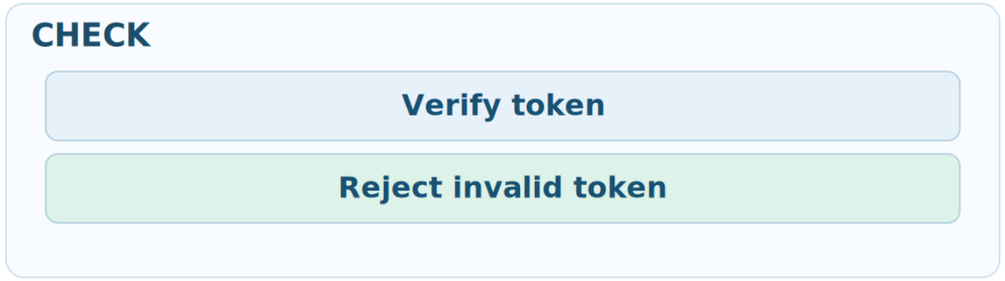
First identify the account and create a single-use reset token.



START; Identify account; Issue reset token

Put password-reset stages in a safe order.

First identify the account and create a single-use reset token.



CHECK; Verify token; Reject invalid token

YOU DO — INDEPENDENT PRACTICE

Questions · Part 1

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

Question 1

Order these steps for ticketing: record purchase, take request, check payment, issue ticket.

Question 2

Design four named subprocedures for booking a classroom and give two key ordering requirements.

WE DO — TRY · DISCUSS · IMPROVE

TRY: Use the original guided questions and diagrams in the preceding We Do pages. DISCUSS: Explain and challenge methods before independent practice. IMPROVE: Note one change now; compare with the source model only AFTER the Exit Ticket.

Guided 1: Predict first Identify the book and verify availability before trying to record a loan.

DISCUSS — Compare the representation or steps you used. Explain one decision and challenge one possible misconception.

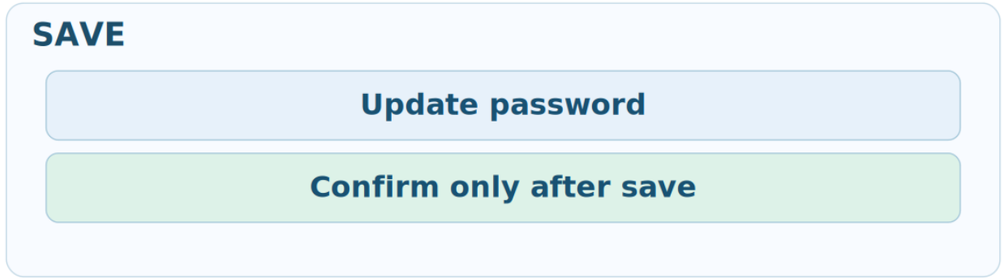
IMPROVE — Write one precise correction or stronger explanation in your own answer.

I Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

Put password-reset stages in a safe order.

First identify the account and create a single-use reset token.



SAVE; Update password; Confirm only after save

WE DO — GUIDED PRACTICE

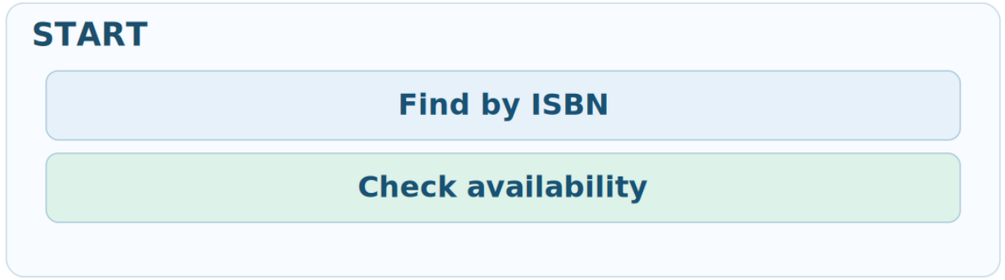
Try → Discuss → Improve

We Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

Order steps for borrowing a library book online.

Identify the book and verify availability before trying to record a loan.



START; Find by ISBN; Check availability